МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення

ЗВІТ

з лабораторної роботи № 1
з дисципліни «Розробка мобільних додатків»
на тему:

«Використання Expo для створення найпростішого додатку React Native. Знайомство з основними компонентами»

Виконала: студентка групи ІПЗ-24-2
Дзядзін Марина Василівна
Перевірив:
Нерода Сергій Іванович

Житомир - 2026

Мета: навчитися створювати та налаштовувати проєкт у середовищі Expo, ознайомитися зі структурою React Native застосунку та опанувати навички роботи з базовими компонентами.

ЗАВДАННЯ

1. Встановити та налаштувати програмне забезпечення для розробки на React Native та запуску застосунку в емуляторі Android (Android Studio) і на фізичному пристрої (Expo Go).
2. Створити найпростіший застосунок на React Native та запустити його на емуляторі й на реальному пристрої.
3. Реалізувати екрани «Головна», «Галерея» та «Профіль», використовуючи StyleSheet API, Flexbox, навігацію Bottom Tabs в Expo Router та іконки з бібліотеки @expo/vector-icons; оформлення виконати згідно з бренд-системою Державного університету «Житомирська політехніка».
4. Розмістити код у репозиторії «MobileLabsRN2026» в окремій папці lab1, додати README.md з описом, інструкцією із запуску, скріншотами та висновками щодо способів запуску застосунку.

ХІД РОБОТИ

1. Підготовка середовища та створення проєкту

Для розробки встановлено Node.js (LTS), Android Studio з віртуальним пристроєм (AVD) та застосунок Expo Go на смартфоні. Встановлена на смартфоні версія Expo Go підтримує Expo SDK 57, тому проєкт створено на шаблоні цієї версії SDK:

Лістинг 1.1 - Створення та очищення проєкту

```
npx create-expo-app@latest lab1 --template default@sdk-57
cd lab1
npm run reset-project
npx expo install @expo/vector-icons @expo-google-fonts/montserrat
```

Команда `npm run reset-project` прибирає демонстраційний код шаблону. Додаткові бібліотеки встановлено через `npx expo install`, яка автоматично підбирає версії, сумісні з поточним Expo SDK. Використані технології: Expo SDK 57, React Native 0.86, React 19.2, Expo Router.

2. Структура проєкту

Expo Router використовує файлову маршрутизацію: кожен файл у папці `app/` є екраном, а файли `_layout.jsx` описують навігатори. Група `(tabs)` об'єднує три вкладки нижньої навігації та не додається до URL. Структура проєкту:

Лістинг 1.2 - Структура проєкту lab1

```
lab1/
├── app/
│   ├── _layout.jsx          # кореневий Stack, шрифти, SafeAreaProvider
│   └── (tabs)/
│       ├── _layout.jsx      # Bottom Tabs + власний хедер
│       ├── index.jsx        # «Головна»: стрічка новин і пошук
│       ├── gallery.jsx      # «Галерея»: сітка / список, перегляд фото
│       └── profile.jsx      # «Профіль»: редагування, налаштування
├── components/
│   ├── AppHeader.jsx        # хедер з кнопкою налаштувань
│   └── BrandMark.jsx        # емблема у фірмових кольорах
├── constants/theme.js       # кольори, шрифти, дані студента
├── data/content.js          # новини та фото галереї
├── assets/images/           # обкладинки новин і фото галереї
├── screenshots/             # скріншоти для README
├── app.json, package.json
└── README.md
```

3. Бренд-система та тема оформлення

Відповідно до бренд-системи університету (media.ztu.edu.ua) використано фірмові кольори: синій `#224A98` і зелений `#4CAD3B` як основні, червоний `#E53E23` та помаранчевий `#EE781C` як акцентні, а також фірмовий шрифт Montserrat. Кольори, шрифти й дані студента винесено в окремий модуль, щоб уникнути дублювання значень у компонентах.

Лістинг 1.3 - constants/theme.js

```
// Бренд-система Державного університету «Житомирська політехніка»
// https://media.ztu.edu.ua/
export const colors = {
  blue: '#224A98', // основний
  green: '#4CAD3B', // основний
  red: '#E53E23', // акцентний
```

```
  orange: '#EE781C', // акцентний

  background: '#F4F6FA',
  surface: '#FFFFFF',
  border: '#E1E6EF',
  text: '#1B2333',
  textMuted: '#6B7486',
  placeholder: '#9AA3B2',
};

export const fonts = {
  regular: 'Montserrat_400Regular',
  medium: 'Montserrat_500Medium',
  semibold: 'Montserrat_600SemiBold',
  bold: 'Montserrat_700Bold',
};

export const student = {
  fullName: 'Дзядзін М. В.',
  group: 'ІПЗ-24-2',
  email: 'ipz242_dmv@student.ztu.edu.ua',
};

export const radius = { sm: 8, md: 12, lg: 16 };
```

4. Кореневий макет і навігація Bottom Tabs

У кореневому макеті завантажуються шрифти Montserrat і підключається `SafeAreaProvider`, який забезпечує коректні відступи від вирізів екрана та статус-бару. Нижню навігацію реалізовано компонентом `Tabs` з Expo Router: для кожної вкладки задано назву та іконку Ionicons (заповнена іконка для активної вкладки, контурна - для неактивних). Через опцію `header` підключено власний компонент хедера.

Лістинг 1.4 - app/_layout.jsx

```
import {
  Montserrat_400Regular,
  Montserrat_500Medium,
  Montserrat_600SemiBold,
  Montserrat_700Bold,
  useFonts,
} from '@expo-google-fonts/montserrat';
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { ActivityIndicator, View } from 'react-native';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { colors } from '../constants/theme';

export default function RootLayout() {
  const [fontsLoaded] = useFonts({
    Montserrat_400Regular,
    Montserrat_500Medium,
    Montserrat_600SemiBold,
    Montserrat_700Bold,
```

```
  });

  if (!fontsLoaded) {
    return (
      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
        <ActivityIndicator color={colors.blue} />
      </View>
    );
  }

  return (
    <SafeAreaProvider>
      <StatusBar style="dark" />
      <Stack screenOptions={{ headerShown: false }} />
    </SafeAreaProvider>
  );
}
```

Лістинг 1.5 - app/(tabs)/_layout.jsx

```
import { Ionicons } from '@expo/vector-icons';
import { Tabs } from 'expo-router';
import AppHeader from '../../components/AppHeader';
import { colors, fonts } from '../../constants/theme';

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        header: () => <AppHeader />,
        tabBarActiveTintColor: colors.blue,
        tabBarInactiveTintColor: colors.textMuted,
        tabBarLabelStyle: { fontFamily: fonts.semibold, fontSize: 11 },
        tabBarStyle: { backgroundColor: colors.surface, borderTopColor: colors.border
},
        sceneStyle: { backgroundColor: colors.background },
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: 'Головна',
          tabBarIcon: ({ color, size, focused }) => (
            <Ionicons name={focused ? 'home' : 'home-outline'} size={size}
color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="gallery"
        options={{
          title: 'Галерея',
          tabBarIcon: ({ color, size, focused }) => (
            <Ionicons name={focused ? 'images' : 'images-outline'} size={size}
color={color} />
          ),
        }}
      />
      <Tabs.Screen
        name="profile"
        options={{
          title: 'Профіль',
          tabBarIcon: ({ color, size, focused }) => (
            <Ionicons name={focused ? 'person' : 'person-outline'} size={size}
color={color} />
          ),
```

```
        }}
      />
    </Tabs>
  );
}
```

5. Хедер застосунку

Хедер містить емблему у фірмових кольорах, назву роботи, ПІБ і групу студента та кнопку налаштувань. Натискання на кнопку відкриває модальне вікно «Про застосунок» (компонент `Modal`). Верхній відступ береться з хука `useSafeAreaInsets()`, тому хедер не перекривається статус-баром.

Лістинг 1.6 - components/AppHeader.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { colors, fonts, student } from '../constants/theme';
import BrandMark from './BrandMark';


export default function AppHeader() {
  const insets = useSafeAreaInsets();
  const [aboutVisible, setAboutVisible] = useState(false);


  return (
    <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
      <BrandMark size={40} />


      <View style={styles.titles}>
        <Text style={styles.title}>Лабораторна робота 1</Text>
        <Text style={styles.subtitle}>
          {student.fullName}, {student.group}
        </Text>
      </View>


      <Pressable
        onPress={() => setAboutVisible(true)}
        style={({ pressed }) => [styles.iconButton, pressed && { opacity: 0.6 }]}
        accessibilityLabel="Налаштування"
      >
        <Ionicons name="settings-sharp" size={22} color="#fff" />
      </Pressable>


      <Modal visible={aboutVisible} transparent animationType="fade"
onRequestClose={() => setAboutVisible(false)}>
        <Pressable style={styles.backdrop} onPress={() => setAboutVisible(false)}>
          <View style={styles.sheet}>
            <Text style={styles.sheetTitle}>Про застосунок</Text>
            <Text style={styles.sheetText}>Дисципліна: Розробка мобільних
додатків</Text>
            <Text style={styles.sheetText}>Тема: Expo та базові компоненти React
Native</Text>
            <Text style={styles.sheetText}>
              Виконала: {student.fullName}, {student.group}
            </Text>
```

```
            <Pressable style={styles.sheetButton} onPress={() =>
setAboutVisible(false)}>
              <Text style={styles.sheetButtonText}>Закрити</Text>
            </Pressable>
          </View>
        </Pressable>
      </Modal>
    </View>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```

6. Екран «Головна»

На головному екрані виводиться стрічка новин університету (компонент `FlatList`). Кожна картка містить обкладинку (`Image`), заголовок, дату та короткий опис; натискання відкриває повну новину на сайті news.ztu.edu.ua у вбудованому браузері (`expo-web-browser`). Над списком розміщено поле пошуку (`TextInput`), яке фільтрує новини за заголовком і описом; для порожнього результату показується повідомлення. Розмір обкладинки обчислюється з ширини екрана (`useWindowDimensions`), що зберігає пропорції 16:9 на будь-якому пристрої.

Лістинг 1.7 - app/(tabs)/index.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import * as WebBrowser from 'expo-web-browser';
import { useMemo, useState } from 'react';
import { FlatList, Image, Platform, Pressable, StyleSheet, Text, TextInput,
useWindowDimensions, View } from 'react-native';
import { colors, fonts, radius } from '../../constants/theme';
import { news } from '../../data/content';


function NewsCard({ item }) {
  const { width } = useWindowDimensions();
  const coverWidth = width - 32; // відступи списку 16 + 16
  return (
    <Pressable
      style={({ pressed }) => [styles.card, pressed && { opacity: 0.85 }]}
      onPress={() => WebBrowser.openBrowserAsync(item.url)}
    >
      <Image
        source={item.image}
        style={{ width: coverWidth, height: (coverWidth * 9) / 16 }}
        resizeMode="cover"
      />
      <View style={styles.cardBody}>
        <Text style={styles.cardTitle} numberOfLines={2}>
          {item.title}
        </Text>
        <Text style={styles.cardDate}>{item.date}</Text>
        <Text style={styles.cardExcerpt} numberOfLines={3}>
          {item.excerpt}
        </Text>
```

```
      </View>
    </Pressable>
  );
}

export default function HomeScreen() {
  const [query, setQuery] = useState('');

  const filtered = useMemo(() => {
    const q = query.trim().toLowerCase();
    if (!q) return news;
    return news.filter((n) => n.title.toLowerCase().includes(q) ||
n.excerpt.toLowerCase().includes(q));
  }, [query]);

  return (
    <FlatList
      data={filtered}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => <NewsCard item={item} />}
      contentContainerStyle={styles.list}
      keyboardShouldPersistTaps="handled"
      ListHeaderComponent={
        <View style={styles.search}>
          <Ionicons name="search" size={18} color={colors.textMuted} />
          <TextInput
            value={query}
            onChangeText={setQuery}
            placeholder="Пошук..."
            placeholderTextColor={colors.placeholder}
            style={styles.searchInput}
          />
          {query.length > 0 && (
            <Pressable onPress={() => setQuery('')} hitSlop={10}>
              <Ionicons name="close-circle" size={18} color={colors.placeholder} />
            </Pressable>
          )}
        </View>
      }
      ListEmptyComponent={<Text style={styles.empty}>Нічого не знайдено за запитом
«{query}»</Text>}
    />
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) стрічка новин б) пошук «Erasmus»

Рисунок 1.1 - Екран «Головна»

7. Екран «Галерея»

Галерея відображає фотографії у двох режимах, які перемикаються кнопками у верхній частині екрана: сітка у дві колонки та список із підписами. Перемикання реалізовано зміною властивості `numColumns` компонента `FlatList`; оскільки ця властивість не може змінюватися «на льоту», списку передається ключ `key={mode}`, що змушує React перемонтувати його. Натискання на фото відкриває повноекранний перегляд у компоненті `Modal`.

Лістинг 1.8 - app/(tabs)/gallery.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { FlatList, Image, Modal, Pressable, StyleSheet, Text, useWindowDimensions,
View } from 'react-native';
import { colors, fonts, radius } from '../../constants/theme';
import { gallery } from '../../data/content';


export default function GalleryScreen() {
  const [mode, setMode] = useState('grid'); // 'grid' | 'list'
  const [selected, setSelected] = useState(null);
```

```
  const isGrid = mode === 'grid';
  const { width } = useWindowDimensions();
  const tileSize = (width - 32 - 12) / 2; // 2 колонки, відступи 16 і проміжок 12
  const viewerSize = width - 32;


  return (
    <View style={{ flex: 1 }}>
      <View style={styles.toolbar}>
        <Text style={styles.heading}>Фотогалерея</Text>
        <View style={styles.switcher}>
          <ModeButton icon="grid" label="Сітка" active={isGrid} onPress={() =>
setMode('grid')} />
          <ModeButton icon="list" label="Список" active={!isGrid} onPress={() =>
setMode('list')} />
        </View>
      </View>

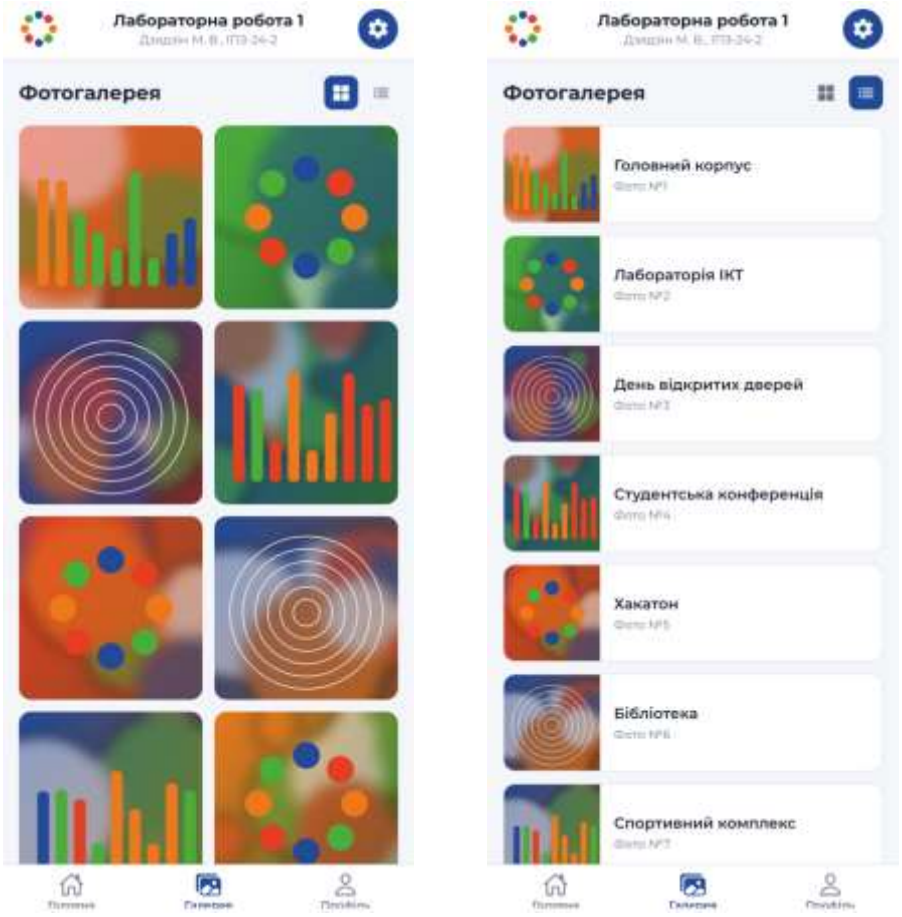

      <FlatList
        key={mode} // зміна numColumns вимагає повторного монтування списку
        data={gallery}
        numColumns={isGrid ? 2 : 1}
        keyExtractor={(item) => item.id}
        contentContainerStyle={styles.list}
        columnWrapperStyle={isGrid ? { gap: 12 } : undefined}
        renderItem={({ item }) => (
          <Pressable
            style={[styles.tile, isGrid ? styles.tileGrid : styles.tileList]}
            onPress={() => setSelected(item)}
          >
            <Image
              source={item.image}
              style={isGrid ? { width: tileSize, height: tileSize } :
styles.imageList}
              resizeMode="cover"
            />
            {!isGrid && (
              <View style={styles.listInfo}>
                <Text style={styles.caption}>{item.caption}</Text>
                <Text style={styles.meta}>Фото №{item.id}</Text>
              </View>
            )}
          </Pressable>
        )}
      />


      <Modal visible={!!selected} transparent animationType="fade" onRequestClose={()
=> setSelected(null)}>
        <Pressable style={styles.viewer} onPress={() => setSelected(null)}>
          {selected && (
            <>
              <Image
                source={selected.image}
                style={{ width: viewerSize, height: viewerSize, borderRadius: 12 }}
                resizeMode="contain"
              />
              <Text style={styles.viewerCaption}>{selected.caption}</Text>
            </>
          )}
        </Pressable>
      </Modal>
    </View>
  );
}


function ModeButton({ icon, label, active, onPress }) {
  return (
    <Pressable onPress={onPress} accessibilityLabel={label} style={[styles.modeButton,
active && styles.modeButtonActive]}>
      <Ionicons name={icon} size={18} color={active ? '#fff' : colors.textMuted} />
```

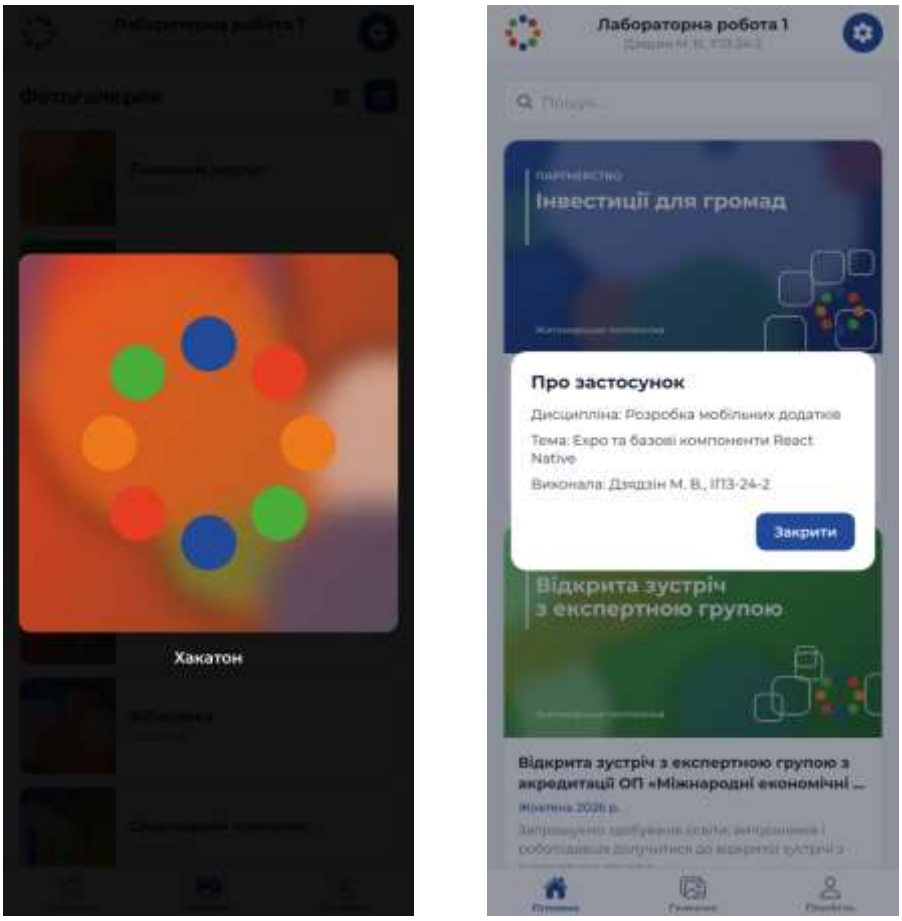
```
    </Pressable>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) режим сітки б) режим списку

Рисунок 1.2 - Екран «Галерея»



а) перегляд фото б) вікно «Про застосунок»

Рисунок 1.3 - Модальні вікна застосунку

8. Екран «Профіль»

Екран профілю складається з кольорового банера, аватара, поля ПІБ та карток «Контакти» і «Налаштування». За замовчуванням поля доступні лише для читання; кнопка з олівцем вмикає режим редагування, у якому з'являються кнопки «Скасувати» (✕) та «Зберегти» (✓). Редагування ведеться в чернетці (`draft`), тому скасування повертає попередні значення, а збереження переносить чернетку в основний стан. Перемикач «Сповіщення» (`Switch`) змінює налаштування одразу. Для коректної роботи з клавіатурою вміст обгорнуто в `KeyboardAvoidingView` та `ScrollView`.

Лістинг 1.9 - app/(tabs)/profile.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { KeyboardAvoidingView, Platform, Pressable, ScrollView, StyleSheet, Switch,
Text, TextInput, View } from 'react-native';
import { colors, fonts, radius, student } from '../../constants/theme';
```

```
const initialProfile = {
  fullName: student.fullName,
  email: student.email,
  group: student.group,
  notifications: true,
};


export default function ProfileScreen() {
  const [profile, setProfile] = useState(initialProfile);
  const [draft, setDraft] = useState(initialProfile);
  const [editing, setEditing] = useState(false);


  const current = editing ? draft : profile;
  const update = (key, value) => setDraft((d) => ({ ...d, [key]: value }));


  const startEdit = () => {
    setDraft(profile);
    setEditing(true);
  };
  const cancel = () => setEditing(false);
  const save = () => {
    setProfile(draft);
    setEditing(false);
  };


  // Перемикачі змінюються одразу, без режиму редагування
  const toggle = (key) => (value) => {
    setProfile((p) => ({ ...p, [key]: value }));
    setDraft((d) => ({ ...d, [key]: value }));
  };


  return (
    <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ?
'padding' : undefined}>
      <ScrollView contentContainerStyle={{ paddingBottom: 32 }}>
        <View style={styles.banner}>
          <View style={styles.bannerActions}>
            {editing ? (
              <>
                <RoundButton icon="close" onPress={cancel} label="Скасувати" />
                <RoundButton icon="checkmark" onPress={save} label="Зберегти" />
              </>
            ) : (
              <RoundButton icon="pencil" onPress={startEdit} label="Редагувати" />
            )}
          </View>
        </View>


        <View style={styles.avatarWrap}>
          <View style={styles.avatar}>
            <Ionicons name="person" size={64} color={colors.textMuted} />
          </View>
        </View>


        <View style={styles.content}>
          <Field label="ПІБ" value={current.fullName} editable={editing}
onChangeText={(v) => update('fullName', v)} />


          <Section title="Контакти">
            <Field
              label="Email"
              value={current.email}
              editable={editing}
              keyboardType="email-address"
              onChangeText={(v) => update('email', v)}
            />
```

```
            <Field label="Група" value={current.group} editable={editing}
onChangeText={(v) => update('group', v)} />
          </Section>


          <Section title="Налаштування">
            <ToggleRow icon="notifications-outline" label="Сповіщення"
value={profile.notifications} onValueChange={toggle('notifications')} />
          </Section>
        </View>
      </ScrollView>
    </KeyboardAvoidingView>
  );
}


function RoundButton({ icon, onPress, label }) {
  return (
    <Pressable onPress={onPress} accessibilityLabel={label} style={({ pressed }) =>
[styles.roundButton, pressed && { opacity: 0.7 }]}>
      <Ionicons name={icon} size={18} color={colors.blue} />
    </Pressable>
  );
}

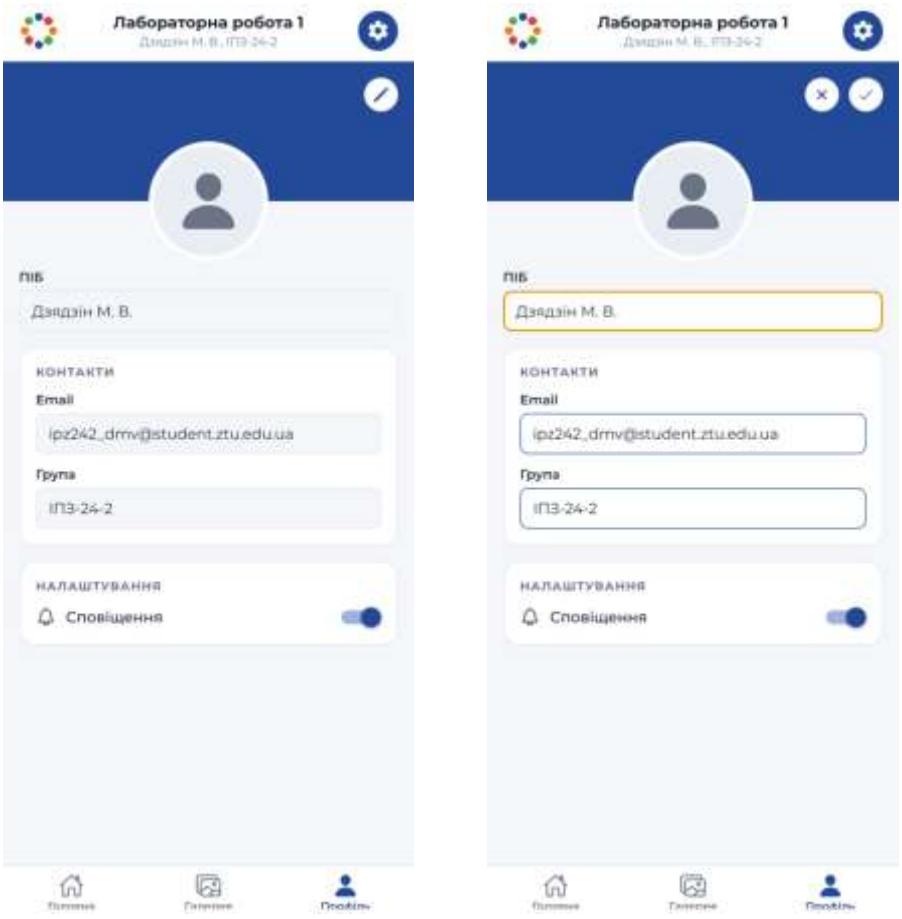

function Section({ title, children }) {
  return (
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>{title.toUpperCase()}</Text>
      {children}
    </View>
  );
}


function Field({ label, editable, ...inputProps }) {
  return (
    <View style={styles.field}>
      <Text style={styles.label}>{label}</Text>
      <TextInput
        {...inputProps}
        editable={editable}
        style={[styles.input, editable && styles.inputEditable]}
        placeholderTextColor={colors.placeholder}
      />
    </View>
  );
}


function ToggleRow({ icon, label, value, onValueChange }) {
  return (
    <View style={styles.toggleRow}>
      <Ionicons name={icon} size={20} color={colors.text} />
      <Text style={styles.toggleLabel}>{label}</Text>
      <Switch
        value={value}
        onValueChange={onValueChange}
        trackColor={{ false: colors.border, true: '#9DB4E3' }}
        thumbColor={value ? colors.blue : '#f4f3f4'}
        activeThumbColor={colors.blue}
      />
    </View>
  );
}


const AVATAR = 120;


// ... оголошення стилів (StyleSheet.create) опущено
```



а) перегляд б) режим редагування

Рисунок 1.4 - Екран «Профіль»

9. Запуск і тестування

Застосунок запускався командою `npx expo start` на фізичному пристрої через Expo Go (сканування QR-коду) та в емуляторі Android (`npx expo start --android`). Перевірено роботу всіх вкладок, пошук і порожній результат пошуку, відкриття новини, перемикання режимів галереї, повноекранний перегляд фото, редагування профілю зі скасуванням і збереженням змін, а також перемикач сповіщень.

Лістинг 1.10 - Команди запуску

```
npx expo start            # сервер розробки, QR-код для Expo Go
npx expo start --android  # запуск з відкриттям в емуляторі Android
npx expo start --clear    # запуск з очищенням кешу Metro
npx expo start --tunnel   # тунельне з'єднання (різні мережі)
```

10. Розміщення коду на GitHub

Вихідний код розміщено у репозиторії https://github.com/ipz242dmv-alt/MobileLabsRN2026 у папці lab1. Файл README.md містить опис проєкту, структуру, інструкцію із запуску, скріншоти та висновки.

ВИСНОВКИ

У ході лабораторної роботи було створено та налаштовано проєкт Expo, прибрано демонстраційний код і реалізовано застосунок із трьома екранами на базових компонентах React Native: `View`, `Text`, `Image`, `TextInput`, `Pressable`, `FlatList`, `ScrollView`, `Switch` і `Modal`. Верстку виконано за допомогою StyleSheet API та Flexbox, навігацію між вкладками - за допомогою Bottom Tabs в Expo Router, де структура маршрутів задається файлами в папці `app/`. Оформлення відповідає бренд-системі Житомирської політехніки.

Під час роботи було розглянуто основні способи запуску мобільного застосунку:

- **Expo Go на фізичному пристрої** (`npx expo start` + QR-код) - найшвидший спосіб побачити застосунок на реальному телефоні без збирання; зміни застосовуються через Fast Refresh. Дає змогу перевірити реальну продуктивність, жести, клавіатуру, вирізи екрана. Обмеження: телефон і комп'ютер мають бути в одній мережі, доступні лише нативні модулі, що входять до Expo Go, а версія SDK проєкту має збігатися з версією, яку підтримує встановлений Expo Go.
- **Емулятор Android (AVD)** (`npx expo start --android`) - дозволяє тестувати різні розміри екранів і версії Android без фізичного пристрою, зручний для налагодження та скріншотів, але навантажує комп'ютер, а продуктивність і сенсори відрізняються від реальних.
- **Тунельне з'єднання** (`npx expo start --tunnel`) - дає змогу підключити телефон з іншої мережі (мобільний інтернет, Wi-Fi з

ізоляцією клієнтів); з’єднання стабільне, але оновлення надходять повільніше.

- **Web-версія** (`npx expo start --web`) - швидка перевірка верстки й логіки в браузері через react-native-web, проте без нативної поведінки компонентів.
- **Онлайн-середовище Expo Snack** - не потребує встановлення ПЗ і підходить для невеликих прикладів, але незручне для повноцінного проєкту.

Отже, для щоденної розробки найзручніше поєднувати Expo Go на реальному пристрої з емулятором Android, а тунельне з’єднання та Snack використовувати, коли локальна мережа або встановлення ПЗ недоступні.